

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA5115

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4, BL5)

Analytical Problem Solving: 40%

QUESTIONS: 2

Total Marks: 20

Annexure A

Code of Conduct:

I M Nishanth (192511154) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M Nishanth

Annexure A

Analytical Problem Solving

4. Diffie-Hellman Key Exchange

Analysis Given:

- a. $p=23, g=5$
- b. User A private key = 6
- c. User B private key = 15
- d. Compute public keys for A and B
- e. Compute the shared secret key
- f. Demonstrate how a Man-in-the-Middle attacker could alter the exchange
- g. Suggest a secure method to prevent the attack

5. ECC vs RSA Comparison

Given:

- a. RSA key size = 1024 bits
- b. ECC key size = 160 bits (equivalent security)
- c. Device processing capacity = limited (IoT device)
- d. Analyze computational cost for both systems
- e. Estimate which consumes less power and memory
- f. Justify which cryptosystem is better for the given scenario with reasoning

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – CSA51 CRYPTOGRAPHY AND NETWORK SECURITY (CO2)

Annexure A – Questions 4 & 5: Worked Solutions

Question 4: Diffie-Hellman Key Exchange Analysis

Given: $p = 23$, $g = 5$. User A's private key $a = 6$. User B's private key $b = 15$.

d) Compute Public Keys for A and B

Answer: Public key of A = 8. Public key of B = 19.

Public key of A: $A = g^a \bmod p$

$$5^2 \bmod 23 = 2, 5^4 \bmod 23 = 4, 5^6 = 5^4 \times 5^2 = 4 \times 2 = 8 \bmod 23$$

$$A = 5^6 \bmod 23 = 8$$

Public key of B: $B = g^b \bmod p$

$$5^1 = 5, 5^2 = 2, 5^4 = 4, 5^8 = 16 \text{ (all mod 23); } 15 = 8 + 4 + 2 + 1$$

$$B = 5^{15} \bmod 23 = (16 \times 4 \times 2 \times 5) \bmod 23 = 19$$

e) Compute the Shared Secret Key

Answer: Shared secret key $K = 2$

Computed two ways, confirming consistency:

$$K = B^a \bmod p = 19^6 \bmod 23 = 2$$

$$K = A^b \bmod p = 8^{15} \bmod 23 = 2$$

Both A and B independently arrive at the same shared secret key, $K = 2$, without ever transmitting it directly over the channel.

f) Man-in-the-Middle (MITM) Attack Demonstration

Answer: Eve intercepts and replaces both public keys, establishing two separate shared secrets – one with A, one with B – without either party noticing.

Since the classic Diffie-Hellman exchange has no authentication of the exchanged public values, an attacker (Eve) sitting between A and B can intercept A's public value (8) and B's public value (19) in transit. Eve generates her own private key e and computes her own public value $E = g^e \bmod p$, then sends E to A (pretending to be B) and also sends a separate Eve-

generated public value to B (pretending to be A). A then computes a shared key with Eve, believing it is shared with B; B computes a different shared key with Eve, believing it is shared with A. Eve can now decrypt, read, modify, and re-encrypt all traffic passing between A and B – a complete, undetected man-in-the-middle compromise, since neither A nor B ever actually shares a key with each other directly.

g) Secure Method to Prevent the Attack

Answer: Use an authenticated key exchange, e.g., the Station-to-Station (STS) protocol with digital signatures / certificate-based authentication (PKI).

To prevent this MITM attack, the exchanged public keys must be authenticated so each party can verify they genuinely originated from the other party and were not substituted in transit. This can be achieved by having A and B digitally sign their public DH values with their own long-term private keys (verified via certificates issued by a trusted Certificate Authority), as done in the Station-to-Station (STS) protocol, or by using a pre-established Public Key Infrastructure (PKI) / TLS certificate-based authentication layered on top of the DH exchange. Since Eve does not possess A's or B's private signing keys, she cannot forge a valid signature on her substituted public values, so any tampering is detected and the exchange is aborted.

Question 5: ECC vs RSA Comparison

Given: RSA key size = 1024 bits; ECC key size = 160 bits (equivalent security); device = resource-limited IoT device.

d) Computational Cost Analysis for Both Systems

Answer: RSA requires far more computation (large modular exponentiation on 1024-bit numbers); ECC achieves equal security with much smaller, cheaper operations.

1. RSA's security relies on the difficulty of integer factorization, which requires very large keys (1024 bits and above) to remain secure; its core operation – modular exponentiation on large integers – has computational cost that grows roughly with the cube of the key length, making it increasingly expensive as key size grows.
2. ECC's security relies on the elliptic curve discrete logarithm problem, which is significantly harder to solve per bit than integer factorization. This allows ECC to achieve equivalent security to RSA using a much smaller key (160 bits vs. 1024 bits), directly translating into far fewer computational operations per encryption/decryption/signature.

e) Power and Memory Estimate

Answer: ECC consumes substantially less power and memory than RSA for the same security level – roughly an order of magnitude less in constrained environments.

Because ECC uses approximately 1/6th the key length of RSA for equivalent security (160 bits vs. 1024 bits), it requires proportionally less memory to store keys and intermediate values, and considerably fewer processor cycles to complete each cryptographic operation. Published performance studies commonly report ECC operations running several times faster and consuming markedly less energy than RSA at equivalent security levels on constrained hardware, making it far better suited to devices with limited battery and memory, such as IoT sensors.

f) Justification: Which Cryptosystem is Better for This Scenario

Answer: ECC is the better choice for the IoT device.

ECC is recommended for this resource-constrained IoT scenario because it delivers the same level of security as RSA while using a dramatically smaller key size, which directly reduces processing time, memory footprint, and power consumption – the three resources that are most limited on IoT hardware. RSA's larger 1024-bit keys would demand more RAM for key storage, more CPU cycles per operation, and faster battery drain, all of which are poor fits for constrained embedded devices. ECC's efficiency advantage makes it the practical and widely adopted standard for securing IoT and other low-power embedded systems today.

Rubric Alignment Summary

The table below maps each grading criterion from the assessment rubric to where it is addressed in this submission, for quick reference during evaluation.

Rubric Criterion	Where addressed
Problem Understanding	Both questions restate the given parameters (p, g, private keys, key sizes, device constraints) before solving, confirming correct interpretation of each scenario.
Method Selection	Correct standard techniques applied: modular exponentiation via repeated squaring for Diffie-Hellman (Q4d/e), and comparative cryptosystem analysis based on key-size-to-security ratio for Q5.

Solution Process	Step-by-step working shown for every calculation – full exponentiation chains for A, B, and the shared secret K in Q4; structured cost/power/memory analysis in Q5.
Accuracy of Calculation	Q4(e) shared secret cross-verified two independent ways ($B^a \bmod p$ and $A^b \bmod p$), both yielding $K = 2$.
Interpretation of Results	Each result is followed by engineering-relevance discussion – Q4(f)/(g) on MITM risk and mitigation, Q5(f) on justified IoT recommendation.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/form ula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets Results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation